САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет программной инженерии и компьютерной техники (ПИиКТ)
Дисциплина: «Расширенный курс по бэкенд-разработке в среде Node.JS»

ОТЧЕТ ПО ДОМАШНЕЙ РАБОТЕ №4
«Технический дизайн микросервисной архитектуры»

Выполнил:
студент гр. К3340 (поток БР1.1)
Галилей Кирилл Дмитриевич

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

==
1. ВАРИАНТ
==
Вариант 1. «Приложение для бронирования столиков в ресторанах»

Ключевая функциональность системы согласно заданию:
- Вход (аутентификация с выпуском JWT-токенов доступа)
- Регистрация (создание аккаунтов клиентов и менеджеров с хэшированием паролей)
- Личный кабинет пользователя (профиль, контактные данные, история)
- Поиск ресторанов с фильтрацией по кухне, расположению и цене
- Страница с информацией о ресторане (меню, фото, отзывы, столики)
- История бронирований пользователя (создание, просмотр, отмена)

==
2. ЗАДАЧА
==
1) Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
2) Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3) Спроектируйте разделение вашей БД на обособленные БД
4) Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5) Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6) В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкенд-приложения на отдельные микросервисы

==
3. ХОД РАБОТЫ
==
3.1. Декомпозиция монолита на микросервисы (Bounded Contexts)
Выделено 4 микросервиса:
1. auth-service (порт 3001) - регистрация, аутентификация, профили, JWT
2. restaurant-service (порт 3002) - каталог ресторанов, залы, столы, меню, отзывы
3. booking-service (порт 3003) - оформление бронирования, проверка слотов, овербукинг
4. notification-service (порт 3004) - асинхронный воркер отправки email/SMS

3.2. Реализация концепции Database-per-service
Разделение на 3 изолированные базы данных PostgreSQL:
- auth_db: таблица users
- restaurant_db: таблицы restaurants, tables, menu_items, reviews
- booking_db: таблица reservations
Межбазовые внешние ключи устранены, используются строковые UUID.

3.3. Модели межсервисного взаимодействия
- Синхронное взаимодействие (HTTP/REST):
  - booking-service -> auth-service: GET /api/v1/auth/internal/verify-user/:id
  - booking-service -> restaurant-service: GET /api/v1/restaurants/internal/tables/:id/check
- Асинхронное взаимодействие (AMQP / RabbitMQ):
  - Exchange: booking_events (topic)
  - Routing keys: booking.created, booking.cancelled

- Очереди: booking_created_queue, booking_cancelled_queue (notification-service)

3.4. Внутренние эндпоинты (openapi-internal.yaml)
1. GET /api/v1/auth/internal/verify-user/:id -> 200 OK ({ id, email, role, isActive }), 404
2. GET /api/v1/restaurants/internal/tables/:id/check -> 200 OK ({ tableId, restaurantId, capacity, isActive }), 404

3.5. Отказоустойчивость и согласованность
- Принцип Eventual Consistency
- Таймауты на синхронных HTTP-вызовах (2с connect, 4с read)
- Персистентность очередей RabbitMQ (durable, persistent) и Dead Letter Queue (DLQ)

3.6. Архитектурная диаграмма
Схема сохранена в файлах microservices.drawio, microservices.svg, microservices.png.

==
4. ВЫВОД
==
В ходе выполнения домашней работы №4 спроектирован детальный технический дизайн декомпозиции монолита на микросервисную архитектуру по принципу Database-per-service. Выделены 4 микросервиса с изолированными базами данных, описана гибридная модель взаимодействия (синхронный HTTP REST для валидации и асинхронный RabbitMQ для событий), разработана спецификация внутренних эндпоинтов openapi-internal.yaml и подготовлена архитектурная диаграмма.